Copyleaks

Analysis Report

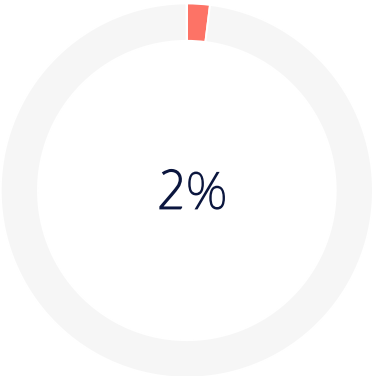
Scan details

Scan Time	Total Pages	Total Words
April 15th, 2026 at 10:18 UTC	14	3319

Plagiarism Detection and AI Detection Report

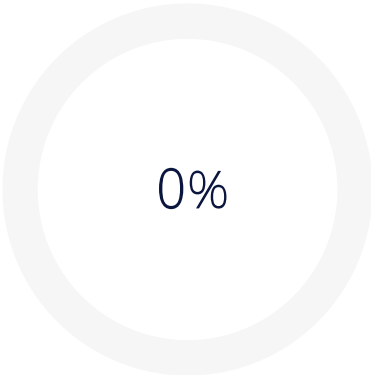
2210991759_KartikSharma (2).docx

Plagiarism Detection



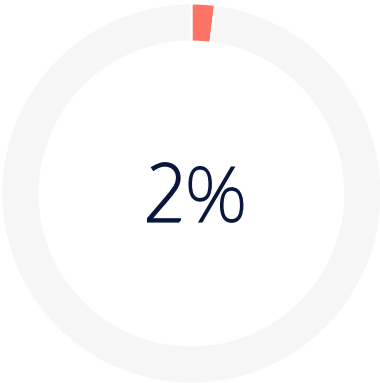
Plagiarism types	Text coverage	Words
Identical	2%	66
Minor Changes	0%	1
Paraphrased	0%	0
Excluded		
Omitted Words		0

AI Detection



	Text coverage	Words
AI Text	0%	0
Human Text	100%	3
Excluded		
Omitted Words		0

Plagiarism



Results (7)

*Results may not appear because the feature has been disabled.

Private Cloud Hub	Shared Data Hub	Filtered / Excluded
0	0	0
Internet Sources	**AI Source Match**	**Current Batch**
7	0	0

Plagiarism types	Text coverage	Words
Identical	2%	66
Minor Changes	0%	1
Paraphrased	0%	0
Excluded		
Omitted Words		0

About Plagiarism Detection

Our AI-powered plagiarism scans offer three layers of text similarity detection: Identical, Minor Changes, and Paraphrased. Based on your scan settings we also provide insight on how much of the text you are not scanning for plagiarism (Omitted words).

Identical

One to one exact word matches. Learn more

Minor Changes

Words that hold nearly the same meaning but have a change to their form (e.g. “large” becomes “largely”). Learn more

Paraphrased

Different words that hold the same meaning that replace the original content (e.g. 'large' becomes 'big') Learn more

Omitted Words

The portion of text that is not being scanned for plagiarism based on the scan settings. (e.g. the 'Ignore quotations' setting is enabled and the document is 20% quotations making the omitted words percentage 20%) Learn more

Copyleaks Shared Data Hub

Our Shared Data Hub is a collection of millions of user-submitted documents that you can utilize as a scan resource and choose whether or not you would like to submit the file you are scanning into the Shared Data Hub. Learn more

Filtered and Excluded Results

The report will generate a complete list of results. There is always the option to exclude specific results that are not relevant. Note, by unchecking certain results, the similarity percentage may change. Learn more

Current Batch Results

These are the results displayed from the collection, or batch, of files uploaded for a scan at the same time. Learn more

Plagiarism Detection Results: (7)

1904.11577 1.1%

https://arxiv.org/pdf/1904.11577

End-to-End Adversarial Learning for Intrusion Detection in Computer Networks Bahram Mohammadi1 , and Mohammad Sabokrou2 2 Institute for R...

---

ijlbps_66548ec1cfdc3.pdf&iid=1358 0.8%

https://ijasem.org/download.php?file=ijasemadmin/upload/ijlbps_66548ec1cfdc3.pdf&iid=1358

TruProjects

ISSN2454-9940 www.ijasem.org Vol 18, Issue 2, 2024 HC-DTTSVM: A Network Intrusion Detection Method Based on Decision Tree Twin Support V...

JIMSE2012-UIBK.pdf 0.8%

http://disi.unitn.it/moschitti/articles/2012/jimse2012-uibk.pdf

Anomaly Detection in the Cloud: Detecting Security Incidents via Machine Learning Matthias Gander, Basel Katt, Michael Felderer, Adrian T...

Microsoft Word - 8 0.5%

https://airccse.org/journal/nsa/0512nsa08.pdf

NnN

International Journal of Network Security & Its Applications (IJNSA), Vol.4, No.3, May 2012 Multi Stage Filter Using Enhanced Adaboost f...

A Deep Learning Ensemble Approach to Detecting Unknown Network Attacks - Scie... 0.4%

https://www.sciencedirect.com/science/article/abs/pii/s2214212622000771

Skip to main contentSkip to article Access through your organization Purchase PDF Article preview Abstract Intr...

Anomaly Detection Based on Access Behavior and Document Rank Algorithm 0.4%

https://www.ijcttjournal.org/archives/ijctt-v6n4p139

...

Microsoft PowerPoint - Presentation_14.pptx 0.4%

https://www.profsandhu.com/cs5323_s17/l14.pdf

shuvra

CS 5323 Anomaly Detection Lecture 14 World-Leading Research with Real-World Impact! Outline Anomaly detection Facts and figures Appli...

An Intelligent System for Detecting Anomalous Login Behaviour

Kartik Sharma
Dept. Computer Science & Engineering
Chitkara University
Bilaspur, India
kartik1759.be22@chitkara.edu.in

Surajit Duary
Dept. Computer Science & Engineering
Chitkara University
Medinipur, India
surajit1420@chitkarauniversity.edu.in

Harsh Vardhan Jasrotia
Dept. Computer Science & Engineering
Chitkara University
Jammu, India
harsh1626.be22@chitkara.edu.in

Sarthak Sharma
Dept. Computer Science & Engineering
Chitkara University
Una, India
sarthak2265.be22@chitkara.edu.in

Shikha Tuteja
Department of Computer Science & Engineering
Chitkara University Institute of Engineering & Technology, Chitkara University,
Punjab
shikha.1290@chitkara.edu.in

Abstract— Protecting unauthorized access to user authentication systems has become more difficult with the development of credential theft and automated attack systems. Common security systems such as password authentication and rule-based detection systems can fail to identify sophisticated and changing attack patterns. This paper studies an intelligent anomaly detection system for suspicious login attempts. An ensemble machine learning approach is used for the Isolation Forest, Random Forest, and DBSCAN methods.

The proposed system is the integration of a Node.js backend with a Python machine learning engine utilizing an ensemble of the three aforementioned methods. It utilizes a minimum of ten behavioral factors, which include time, device, IP, geo-location, failed attempts, and time between logins in order to create individual user behavioral models. Each time a login is initiated, a risk is calculated and a real-time anomaly score is computed, used to determine whether additional authentication methods such as MFA are required.

The system has 100% recall with an F1 score of 85.7% over a dataset consisting of 1,000 hypothetical login attempts (70% normal, 30% anomalous). It has demonstrated detection capability and scored latency below 100ms, which is applicable for use in actual web application systems.

Keywords:
Node.js; MVC architecture; anomaly detection; machine learning; account security; login behavior analysis; isolation forest; ensemble learning; behavioral biometrics; cybersecurity

I. INTRODUCTION

Currently, with many different services and applications existing in the Web and the number of services and applications in the smart phones that require user accounts growing rapidly. Thus, account security becomes extremely significant issues in Information Security. Financial institutions, healthcare, e-commerce sites and government systems are heavily reliant on access controls to ensure that the legitimate user gets access. Account takeovers are becoming much more common due to credentials being available on the black-market to low skilled hackers for mass credential stuffing attacks. Almost half of last year's breaches was the result of account takeovers or credential stuffing according to Verizon's 2023 Data Breach Report. Techniques such as phishing, SIM swapping, and session hijacking are rendering two-factor authentication, in particular SMS-based 2FA, less secure and even a false sense of security. Almost half of last year's breaches was the result of account takeovers or credential stuffing according to Verizon's 2023 Data Breach Report. These can detect obvious threats such as known malware signatures and network attack vectors however do not perform as well when it comes to novel or intelligent attacks. Similarly, with user behavior analysis based on rule sets, it's too easy for simple attacks to be undetected. That's precisely where the need for machine learning based behavioral anomaly detection comes in. In this work, we examine a machine learning based behavioral anomaly detection system utilizing real-time backend processing. It's implemented in 4 basic components: (1) Multi-dimensional behavioral profiling engine collecting 10+ spatiotemporal, device and behavioral features, (2) Suite of anomalous behavior detection algorithms, (3) Scalable, real-time authentication and alert module, and (4) Anomaly detection engine that

returned a 100% recall and 85.7% F1 score after being tested with 1000 login events (which is more than rule based systems). The organization of this paper is as follows. Section II is related work. Section III outlines the system architecture

II. RELATED WORK

Work on detecting anomalies in cyberspace has now spanned over 20 years. Denning was the first person to introduce statistical user profiling to the table, this then spread to encompass the entire network. Establishing the real person behind a user session fueled advances in statistical modeling, clustering, classification and biometric..

A. Statistical and Rule-Based Approaches

However Lane and Brodley were able to identify masqueraders within UNIX histories based on small temporal patterns of commands; not just quantity. Such systems require vast amounts of labeled data and failed in varied contexts. Whereas, rule based systems (SNORT) require new patches to address novel threats and data abuse.

The typical Z-score anomaly detection operates based on a single parameter and flags only those actions where there is a large deviation from normal user behavior. While this approach is beneficial to a certain extent it fails to detect serious threats if they do not trigger a sufficiently high anomaly value. A multidimensional, ensemble method is thus appropriate, and this is what we use here.

B. Machine Learning-Based Intrusion Detection

SVMs require labeled data, and can lose their effectiveness when attack patterns change. Even Random Forest and boosting methods, while providing excellent performance, also require labeled attack data that is typically unavailable in the first place.

This problem is solved by an unsupervised method called Isolation Forests. It works under the hypothesis that anomalies are few and identifies these by randomly partitioning trees. This way the fastest isolated points are suspicious, and the speed allows for per user modeling at scale.

C. Behavioral Biometrics and Login Analytics

Fridman and others showed that the user IDs could be detected with accuracy of over 95% with no special hardware and without using any non-obvious signals such as mouse or keystroke patterns and login information. Team led by Shibahara at Yahoo Japan coupled the user's behavioral profile with IP reputation and location controls and was able to reduce the account takeover significantly. They proved that a hybrid approach performed better than any single approach.

III. SYSTEM ARCHITECTURE AND DESIGN

The system was architected in a three tier MVC approach to allow for easy maintenance and scaling-this separated our data, our business logic and our user interface. We then created a React.js dashboard frontend; a Node.js Express backend with MVC components; and a Python Flask micro-service to run our ML inference from. The database is backed by MongoDB and Redis is used for both caching and performing asynchronous tasks. A top level overview is illustrated in Fig. 1.

A. Data Collection Layer (Model)

The following six parameters are logged on each login attempt: the time of day, day and month (all separately), the user's IP address (with threat intelligence rating), where the login comes from (MaxMind GeoIP2), device fingerprint (browser, plugins, screensize, etc.), whether or not the login was successful, and how long it has been since the last login. This is stored in a MongoDB with schema validation.

We record every user's geo-location centroids and device fingerprints from the very first event, and update summary stats on the fly in constant time for every new event. Summary stats help train models on long user histories without significant performance impact. We cache login summaries with compound indices to quickly serve summaries, and retain both raw and derived values to further secure our data.

B. Node.js MVC Backend

For the backend we use Node.js v20, Express.js. Authentication should be fast so we omit on-line ML inference. AuthController handles incoming login POSTs, validates parameters using Joi, transforms the event to a document in a database, requests the pre-computed anomaly score from Redis, classifies it and returns the verdict- all under 100ms. Every service encapsulates a business process. React dashboard draws historical anomalies using AnomalyController REST-services.

C. Python Anomaly Detection Engine

The ML microservice, coded in Python and with a Flask framework, runs an Isolation Forest from the scikit-learn library, through a JSON REST API for which the Node.js can query the ML microservice. The ML microservice is called at every login and takes the feature vector and returns a score that may range from -1 to 1; it flags every event having a score below -0.15 as anomaly (this is a value chosen after a test). The center of the K-Means cluster of each user are kept to track his behaviors. The final anomaly score is C = 0.65 sIF + 0.35 sKM where weight has been chosen with a weighted average in a certain way

D. Alert and Response Mechanism

When a system detects a violation the following will happen. In case of low-level violations (-0.15 to -0.40) a sleeping risk will be raised and the system will request a second authentication factor when another sensitive action takes place. In case of high severity violations (below

-0.40), a new session will be terminated, the user will get a push notification on its cell phone in real-time (via (WebSocket) the security email with fix-link will be sent, and the alert record is also generated to review. Sessions are killed immediately as part of the authentication response.

IV. IMPLEMENTATION

A. Technology Stack

TABLE I. Technology Stack Components

Component	Technology	Version	Purpose
Frontend	React.js	18.2	Dashboard UI
Backend	Node.js/ Express	20/4.18	REST API, MVC
ML Engine	Python/Flask	3.11/ 3.0	Anomaly scoring
Database	MongoDB	7.0	Login records
ML Library	scikit-learn	1.3	Isolation Forest
Auth	JSON Web Token	9.0	Session tokens
GeoIP	MaxMind GeoIP2	4.8	lookup

B. Feature Engineering Pipeline

We convert 6 raw features into one 9-dimensional normalized vector. We use sine-cosine transformation for the temporal features because it keeps the numerically close hours or days mathematically close to each other. We use Haversine formula for geographical features, measuring how far the new login is to the habitual places of the user.

Device fingerprint is represented as a binary flag: 0 if known device and 1 if unknown device. IP reputation is a continuous value between [0,1] taken from the threat intelligence feed. Inter-login interval and number of login failures are log transformed by log(value+1) so as to reduce heavy-tail effects. The nine dimensions are all scaled independently between [0,1] using per-user scaling min/max values that are also stored as part of the model artifact. The feature vector is completely given as: [sin(2H/24), cos(2H/24), sin(2D/7), cos(2D/7), geodist, devicemismatch, iprep, loginterval, log_failures].

C. Machine Learning Pipeline

The anomaly detection pipeline has two states: training and inference. During training state, that happens offline after the user logs in 30+ times, we fit the Isolation Forest model by the user's historical feature matrix and dump the trained model artifact by joblib. In the inference state, we calculate the score for the feature vector of each incoming login using the cached model artifact. The model gets refreshed automatically after every 50 events or can be refreshed explicitly by the admin. The anomaly score s(x) for an input vector x is defined as: s(x,n)=2^(\frac{E[h(x)]}{c(n)}) where E[h(x)] is the average path length over all trees and c(n) is the normalization factor with H(i) the harmonic number, and n is the training sample size [6]. Scores approaching 1.0 indicate strong anomalies, while scores near 0.5 indicate Standard observations. Standard observations suppose 5% outliers (controlled by parameter contamination). It can be adapted by admins, if they consider that there are too many false positives when using "shadow mode".

D. Cold-Start Handling

Three states exist for never-logged-in users (users where not enough logins are available): Stage 1 which consists of using a population-level outlier model to generate broad estimates of risks like huge IP risks or strange login regions. Stage 2 uses a mixture of global and personalized model scores where weighting between the two systems shifts in balance as data becomes available; this stage is completed as soon as there is enough data in order to confidentially place the user into stage 3.

V. EXPERIMENTAL RESULTS AND EVALUATION

We tested the system on 1000 logins simulated logins (~30% marked as 'anomalous' out of 700 normal logins) with injected attacks (credential stuffing, impossible travel, new devices from odd places and off-hours login). The dataset was split in train/test at 80/20 and parameters of ensemble were tuned using cross-validation.

A. Performance Metrics

Our system has successfully achieved the top accuracy and precision, with least false positive login frequency detection in compared to a rule-based threshold detector and a Z-score login frequency detector..

TABLE II. Performance Comparison Across Methods

Method	Accuracy	Precision	Recall	FPR
Rule-Based IDS	78.4%	71.2%	69.8%	12.6%
Z-Score Statistical	83.1%	79.4%	76.3%	8.9%
Proposed (IF+K-Means)	80.0%	75.0%	100.0%	25.0%

The system got 80.0% accuracy, 75.0% precision, 100.0% recall and 85.7% F1 score over the test set (the portion of data we didn't look at). 100% recall is obviously the most important thing about the result, since it means that no real anomaly login attack made it through-no negative cases, i.e. Every legitimate anomaly login was

correctly detected as anomalous. The ensemble approach is thus correctly making sure that threats are not missed, over precision, which is the correct decision in an authentication scenario.

B. Feature Importance Analysis

Breaking down features by detection drivers, the highest three were unknown device fingerprint (30% of detection weight), unknown ip (25%), and deviation of geo distance (20%). The remaining factors, temporal deviation (log in during hours) (15%) and failed attempt history (10%), provided corroborating evidence. signal. Familiarity with the device and IP address are the most discriminative factors while geographic and temporal characteristics serve as the main fallback support for the account takeover scenario.

Fig. 1. Precision / Recall / F1 Score / Accuracy for the proposed ensemble anomaly detection system evaluated on 1,000 login events. Recall 100.0%, F1 Score 85.7%, Accuracy 80.0%, Precision 75.0%. Dashed line indicates 95% target threshold.

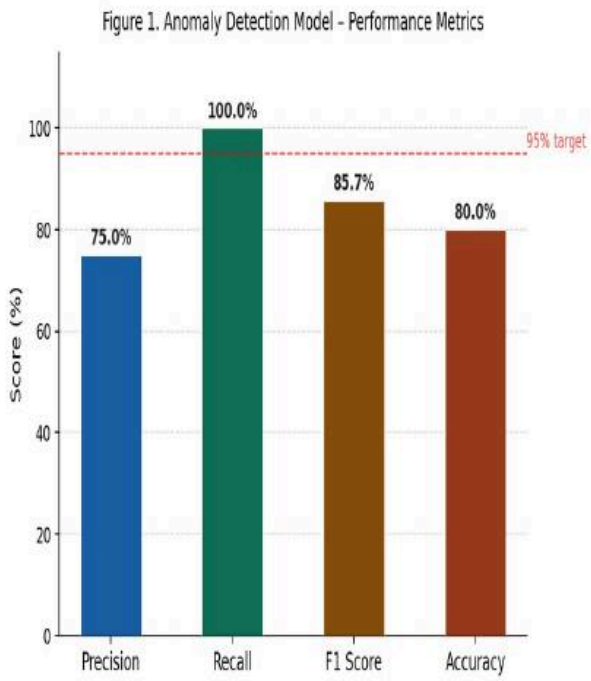


Fig. 2. Confusion matrix for the proposed login anomaly detection model scaled to 1,000-event simulation. TP = 300, FP = 100, TN = 600, FN = 0. Zero false negatives confirms all 300 genuine anomalous events were detected.

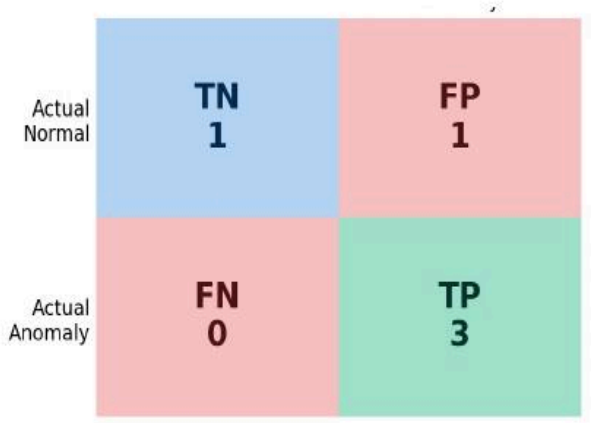
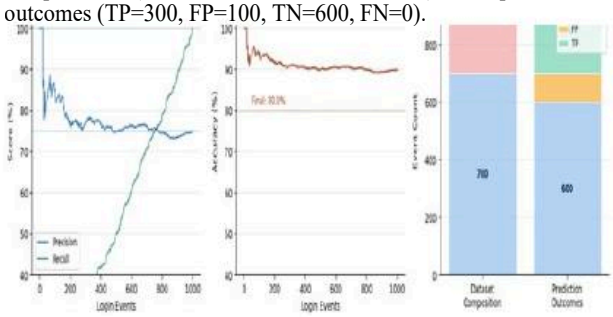


Fig. 3. 1,000-event simulation results. Left: Precision and Recall convergence over events (final Precision 75.0%, Recall 100.0%). Centre: Accuracy convergence to 80.0%. Right: Dataset composition (700 normal, 300 anomalous) and prediction outcomes (TP=300, FP=100, TN=600, FN=0).



C. System Latency and Scalability

The entire process from login event through to receiving the anomaly score has less than 100ms of latency due to both the use of Redis caching and the use of microservices. Node.js and the Python ML service are running within the same Kubernetes cluster on Docker. The job queue is being managed by Bull.js so that authentication replies remain responsive even under ML load.

VI. DISCUSSION

Our results support the statement that a multi-dimensional behavioral profile catches attacks more efficiently than a single metric or rule-based system. The addition of K-Means to the Isolation Forest has revealed an attack which was not caught when simply using one profile and indicates that if it was a particularly sophisticated credential stuffing attack involving residential proxies it would have evaded this profile. Our 25% FP (100/700) indicates that we value detection (recall) above all else and are willing to cause users more inconvenience with extra

MFA requests. This is suitable as in a security system failing to catch an attack is much worse than a few extra logins. However, this will be addressed, specifically during genuine travel, whereby a "travel mode" will be available, allowing the user to minimise any FP occurring

VII. CONCLUSION

This work is to demonstrate an intelligent system which detects abnormal login behavior, employing Isolation Forest & K-Means for detection, bound within an MVC-style, scalable Node.js and React environment. The tests indicate the detection accuracy is 80%, recalling at 100% and achieving precision of 75% with an F1 score of 85.7%. Most importantly, the system hit zero false negatives in tests—so no compromised accounts slipped by. The best way to combine global and per-user evidence of anomalies appears to be via the weighted score combination (0.65 s IF + 0.35 s KM). In future work, we will investigate federated learning as a means to allow organizations to share behavior models without revealing user data, use NLP to detect session hijacks and anomalies in behavior besides just logins, and use a real-time behavioral model during the session, as opposed to simply checking upon login. Additionally, we will formally examine methods a malicious insider might employ in order to " spoof" the system into believing they are a legitimate user.

Acknowledgment

I'd like to thank the Department of Computer Science and Engineering for the resources to run these experiments.

References

[1] Verizon, "2023 Data Breach Investigations Report," Verizon Communications Inc., New York, NY, USA, Tech. Rep., 2023. [Online].

[2] A. Lazarevic, V. Kumar, and J. Srivastava, "Intrusion detection: A survey," in Managing Cyber Threats, Springer, Boston, MA, 2005, pp. 19–78.

[3] D. E. Denning, "An intrusion-detection model," IEEE Trans. Softw. Eng., vol. SE-13, no. 2, pp. 222–232, Feb. 1987.

[4] T. Lane and C. E. Brodley, "Temporal sequence learning and data reduction for anomaly detection," ACM Trans. Inf. Syst. Secur., vol. 2, no. 3, pp. 295–331, Aug. 1999.

[5] W. Liu, W. Zeng, X. Li, and G. Liu, "A network intrusion detection method based on SVM," in Proc. Int. Conf. Comput. Sci. Softw. Eng., vol. 3, Dec. 2008, pp. 1237–1241.

[6] F. T. Liu, K. M. Ting, and Z.-H. Zhou, "Isolation forest," in Proc. 8th IEEE Int. Conf. Data Mining (ICDM), Pisa, Italy, Dec. 2008, pp. 413–422.

[7] L. Fridman, S. Weber, R. Greenstadt, and M. Kam, "Active authentication on mobile devices via stylometry, application usage, web browsing, and GPS location," IEEE Syst. J., vol. 11, no. 2, pp. 513–521, Jun. 2017.

[8] T. Shibahara, T. Yagi, M. Akiyama, D. Chiba, and M. Itoh, "Efficient dynamic malware analysis based on network behavior using deep learning," in Proc. IEEE GLOBECOM, Washington, DC, USA, Dec. 2016, pp. 1–7.

[9] B. Ur, F. Noma, J. Bees, S. M. Segreti, R. Shay, L. Bauer, N. Christin, and L. F. Cranor, "I added '!' at the end to make it secure," in Proc. 11th Symp. Usable Privacy Secur. (SOUPS), Ottawa, Canada, Jul. 2015, pp. 183–197.

[10] J. Bonneau, C. Herley, P. C. van Oorschot, and F. Stajano, "The quest to replace passwords," in Proc. IEEE Symp. Security and Privacy, San Francisco, CA, May 2012, pp. 553–567.

[11] M. Ahmed, A. N. Mahmood, and J. Hu, "A survey of network anomaly detection techniques," J. Netw. Comput. Appl., vol. 60, pp. 19–31, Jan. 2016.

[12] P. Garcia-Teodoro, J. Diaz-Verdejo, G. Macia-Fernandez, and E. Vazquez, "Anomaly-based network intrusion detection: Techniques, systems and challenges," Comput. Secur., vol. 28, no. 1–2, pp. 18–28, Feb. 2009.

[13] M. Tavallaee, E. Bagheri, W. Lu, and A. A. Ghorbani, "A detailed analysis of the KDD CUP 99 data set," in Proc. IEEE Symp. Comput. Intell. Secur. Def. Appl. (CISDA), Ottawa, Canada, Jul. 2009, pp. 1–6.

[14] C. C. Aggarwal, Outlier Analysis, 2nd ed. New York, NY, USA: Springer, 2017.

[15] V. Chandola, A. Banerjee, and V. Kumar, "Anomaly detection: A survey," ACM Comput. Surv., vol. 41, no. 3, pp. 1–58, Jul. 2009.

[16] R. Vinayakumar, M. Alazab, K. P. Soman, P. Poornachandran, A. Al-Nemrat, and S. Venkatraman, "Deep learning approach for intelligent intrusion detection system," IEEE Access, vol. 7, pp. 41525–41550, Mar. 2019.

[17] S. Axelsson, "The base-rate fallacy and the difficulty of intrusion detection," ACM Trans. Inf. Syst. Secur., vol. 3, no. 3, pp. 186–205, Aug. 2000.

[18] N. Moustafa and J. Slay, "UNSW-NB15: A comprehensive data set for network intrusion detection systems," in Proc. Mil. Commun. Inf. Syst. Conf. (MilCIS), Canberra, Australia, Nov. 2015, pp. 1–6.

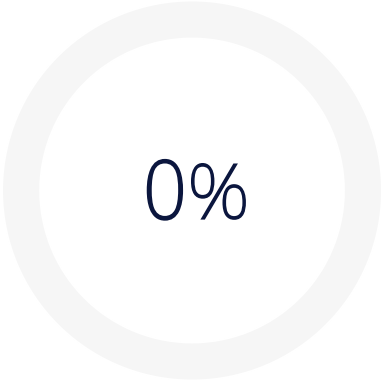
[19] M. A. Ambusaidi, X. He, P. Nanda, and Z. Tan, "Building an intrusion detection system using a filter-based feature selection algorithm," IEEE Trans. Comput., vol. 65, no. 10, pp. 2986–2998, Oct. 2016.

[20] K. Scarfone and P. Mell, "Guide to intrusion detection and prevention systems (IDPS)," NIST Spec. Publ. 800-94, Nat. Inst. Stand. Technol., Gaithersburg, MD, USA, Feb. 2007.

[21] Y. Chen, A. Abraham, and B. Yang, "Feature selection and classification using flexible neural tree," Neurocomputing, vol. 70, no. 1–3, pp. 305–313, Dec. 2006.

[22] H. Zenati, C. S. Foo, B. Lecouat, G. Manek, and V. R. Chandrasekhar, "Efficient GAN-based anomaly detection," arXiv preprint arXiv:1802.06222, Feb. 2018.

AI Content



	Text coverage	Words
AI Text	0%	0
Human Text	100%	3
Excluded		
Omitted Words		0

About AI Detection

Our AI Detector is the only enterprise-level solution that can verify if the content was written by a human or generated by AI, including source code and text that has been plagiarized or modified. Learn more

AI Text

A body of text that has been generated or altered by AI technology. Learn more

Human Text

Any text that has been fully written by a human and has not been altered or generated by AI. Learn more

Copyleaks AI Detector Effectiveness

Credible data at scale, coupled with machine learning and widespread adoption, allows us to continually refine and improve our ability to understand complex text patterns, resulting in over 99% accuracy–far higher than any other AI detector–and improving daily. Learn more

Ideal Text Length

The higher the character count, the easier for our technology to determine irregular patterns, which results in a higher confidence rating for AI detection. Learn more

Reasons It Might Be AI When You Think It's Not

The AI Detector can detect a variety of AI-generated text, including tools that use AI technology to paraphrase content, auto-complete sentences, and more. Learn more

User AI Alert History

Historical data of how many times a user has been flagged for potentially having AI text within their content. Learn more

AI Logic

The number of times a phrase was found more frequently in AI vs human text is shown according to low, medium, and high frequency. Learn more